

ECE573: Advanced Embedded Logic design

Quiz 1: Understand the basics of APB, ATB, AHB, and AHB-Lite Protocol along with evolution in AMBA 1-5

Group No-4 Members

Nakul Garg	Ankit Kumar Pal	Shivam Shukla	Anshu Kumar	Dora Sai Kumar
2022309	2021132	2022309	PhD24107	22DR0293

Index

1. Introduction to Advanced Microcontroller Bus Architecture
2. Advanced Peripheral Bus (APB)
3. Advance Trace Bus (ATB)
4. Advanced High - performance Bus (AHB)
5. Advance High - performance Bus Lite (AHB lite)
6. Comparison
7. Contribution
8. Reference

Introduction to Advanced Microcontroller Bus Architecture

Introduction:

The Advanced Microcontroller Bus Architecture (AMBA) is the de facto standard for on-chip communication. AMBA is used to design high-performance embedded microcontrollers. Arm develops the specification and is an open standard for the chip design industry.

To simplify chip design integration, AMBA protocols have these features:

- Synchronous operations: Using only the clock's rising edge makes working with standard synthesis tools easy.
- No bi-directional signals: Avoid tri-state buffers, simplifying on-chip signal management.

Following are the AMBA bus protocols that are used in microcontrollers:

- Advanced System Bus (ASB)
- Advanced Peripheral Bus (APB)
- Advanced Trace Bus (ATB)
- Advanced High-Performance Bus (AHB)
- Advanced High-Performance Bus Lite (AHB-Lite)
- Advanced eXtensible Interface (AXI)

Evolution of AMBA 1-5:

Bus Protocol	AMBA 1	AMBA 2	AMBA 3	AMBA 4	AMBA 5	Notes
ASB	ASB	ASB				Used on ARM7TDMI, obsolete
APB	APB	APB2	APB3	APB4		Used on almost all ARM processor systems
ATB			ATB			For transferring trace data during debugging
AHB		AHB	ABH-lite		AHB5	Used on the Cortex-M processors
AXI			AXI3	AXI4 AXI-lite AXI-stream	AXI5	Used on a high-performance processor
ACE				ACE ACE-Lite	ACE5 ACE5-Lite	Used on high-performance with cache coherency requirement
CHI					CHI	Advanced coherency management
DTI					DTI	Used with system-level memory management unit (MMU)
Low-Power Interface Specification				Q channel and P channel		For power management

Advanced Peripheral Bus

Introduction:

The Advanced Peripheral Bus (APB) is a key component of the Advanced Microcontroller Bus Architecture (AMBA) protocol family, designed to provide a low-cost interface optimized for low power consumption and reduced complexity.

APB was introduced in the AMBA 2 specification, primarily intended for peripheral connections. Subsequent enhancements in AMBA 3 and AMBA 4 added support for wait states, error responses, and advanced transfer attributes, including TrustZone support. While the APB protocol does not limit bus width, most Arm-based systems commonly adopt a 32-bit peripheral bus configuration.

The objective of APB:

The Advanced Peripheral Bus (APB) is designed with the following key objectives:

1. Low-Power Consumption:
2. Easy interfacing with other Bus protocol
3. Two-cycle transfer operation for smooth transition

The Evolution of APB:

The APB Specification Rev E, released in 1998, is now obsolete and is superseded by the following three revisions:

- **AMBA 2 APB Specification:** Added basic read/write transfers (**APB2**).
- **AMBA 3 APB Protocol Specification v1.0:** Added wait states and error reporting and supported the functionalities PREADY (indicates transfer completion) and PSLVERR (indicates transfer failure) (**APB3**).
- **AMBA APB Protocol Specification v2.0:** Added transaction protection and sparse data transfer, supported the functionalities PPROT (supports secure/non-secure transactions) and PSTRB (enables sparse data transfer) (**APB4**).

APB signals & connection: P is the signal prefix that indicates an APB signal.

Signal	Source & Destination	Description
PCLK	Clock Source → All APB blocks	Clock signal
PSEL	Address decoder → Slave	Device select
PADDR[n:0]	Master → Slave	Address bus
PENABLE	Master → Slave	Transfer control
PWRITE	Master → Slave	Write control (1=Write, 0=Read)
PWDATA[31:0]	Master → Slave	Write data

PRDATA[31:0]	Master ← Slave	Read Data
PREADY	Master ← Slave	Slave ready (transfer completed, AMBA 3 and onwards)
PSLVERR	Master ← Slave	Slave response (AMBA 3 and onwards)
PSTRB[n-1:0]	Master → Slave	Byte strobe for write operations (AMBA 4)
PPROT[2:0]	Master → Slave	Transfer Protection control (AMBA 4)

APB occupies a small memory space with an address bus typically less than 32 bits. All transfers are 32-bit, with no transfer size control. The two least significant bits of PADDR are unused for word alignment.

Data buses:

The APB protocol features two separate data buses: one for read operations and another for write operations. Each bus can support widths of up to 32 bits. However, since these buses lack dedicated handshake signals, simultaneous data transfers on both buses are not possible.

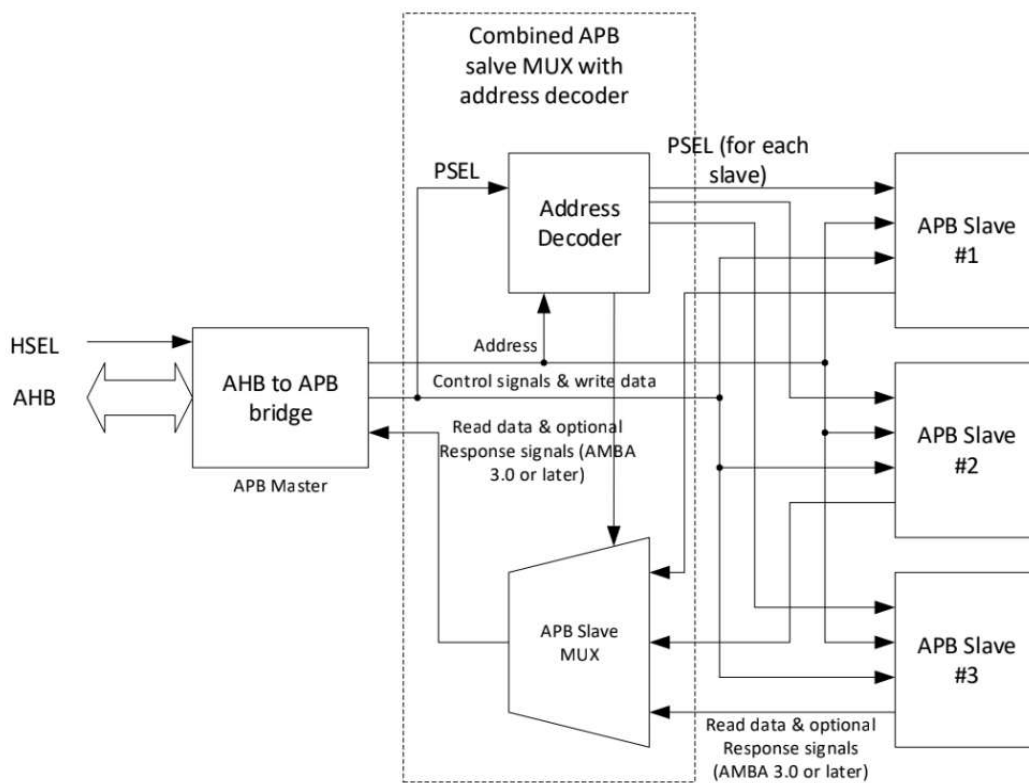


Fig.1: An example APB subsystem.

The AHB to APB Bridge interfaces the Advanced High-performance Bus (AHB) to the Advanced Peripheral Bus (APB). It converts control signals and data from the AHB into a format compatible with the APB protocol. It acts as the master for all connected APB slaves.

The address decoder via PSEL signal selects the slave for the given transaction, and the selected slave executes the operation and sends read data & optional response signals to the master through the APB slave MUX. APB slave MUX takes read data and optional response signals from the selected APB slave and forwards them to the APB master. Unlike AHB, APB operations are not pipelined. In AMBA 2, APB transfers must be two cycles to ensure proper communication.

In AMBA 2, APB transfers are designed to occur over two cycles to ensure proper communication. The first cycle latches the address and control signals, while the second cycle is dedicated to data transfer. This two-cycle structure allows the peripheral sufficient time to process the address before receiving the data, enhancing clarity and reliability in the transfer process.

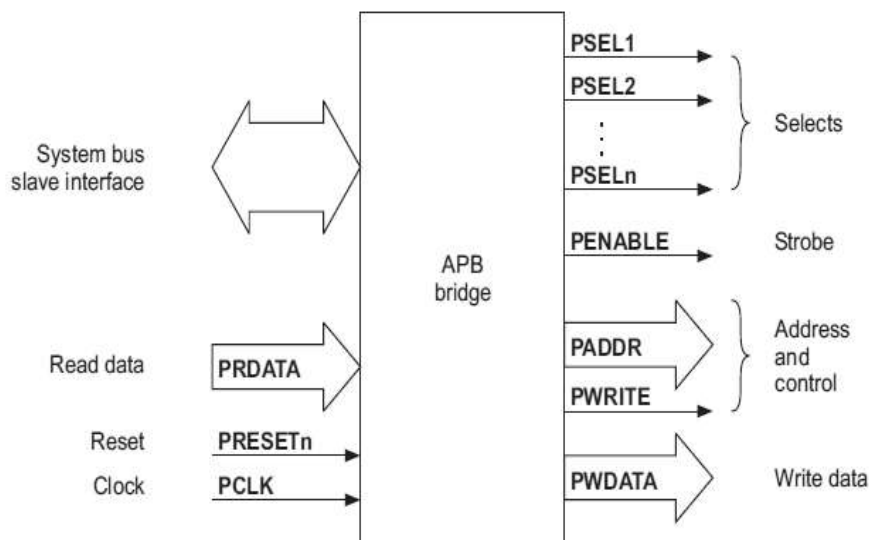


Fig.2: APB bridge interface diagram

APB bridge: The APB bridge is the only bus master on the AMBA APB. In addition, the APB bridge is also a slave on the higher-level system bus.

APB bridge description: The bridge unit converts system bus transfers into APB transfers and performs the following functions:

- Latches the address and holds it valid throughout the transfer.
- Decodes the address and generates a peripheral select, PSELx. Only one select signal can be active during a transfer.
- Drives the data onto the APB for a write transfer.
- Drives the APB data onto the system bus for a read transfer.
- Generates a timing strobe, PENABLE, for the transfer.

APB write transfer:

With no wait state:

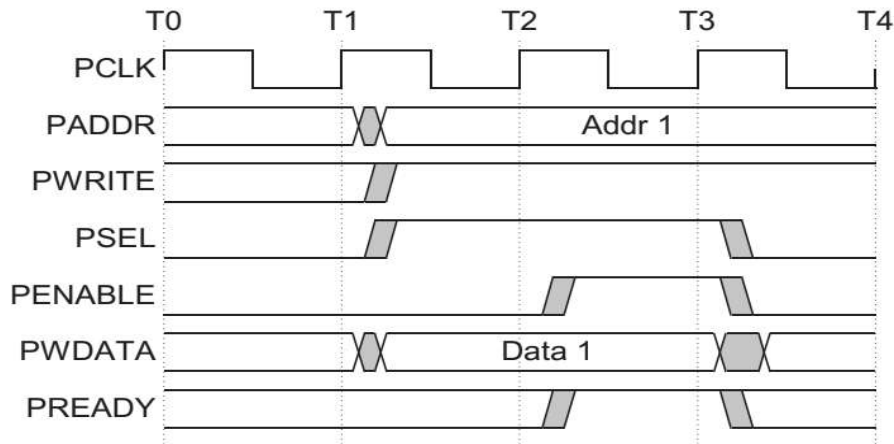


Fig.3: Write transfer with no wait state

- At time T1, a write transfer begins when the address (PADDR), write data (PWDATA), write signal (PWRITE), and select signal (PSEL) are captured on the rising edge of PCLK. This phase is referred to as the Setup phase of the write transfer.
- At T2, enable signal PENABLE and ready signal PREADY are registered at the rising edge of PCLK.
- When asserted, PENABLE signifies the beginning of the Access phase of the transfer.
- When PREADY is asserted, it indicates that the slave is ready to complete the transfer on the next rising edge of PCLK.
- The address (PADDR), write data (PWDATA), and control signals remain valid until the transfer concludes at T3, marking the end of the Access phase.
- The enable signal PENABLE, is deasserted at the end of the transfer. The select signal PSEL, is also deasserted unless the transfer is to be followed immediately by another transfer to the same peripheral.

With wait state:

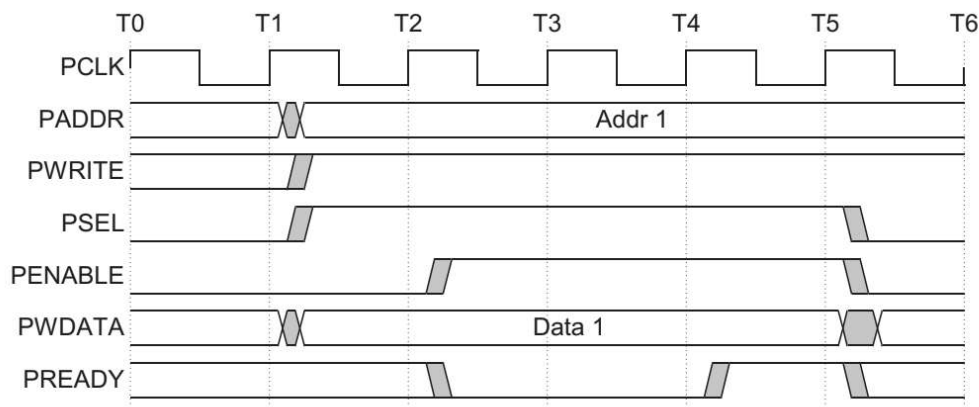


Fig.4: Write transfer with wait state

PREADY signal is used to extend the transfer by the slave. During an Access phase, when PENABLE is HIGH, the slave extends the transfer by driving PREADY LOW.

To reduce the low power consumption, it is highly recommended that the address and write signals aren't changed immediately after the transfer but remain stable until access occurs.

Write strobes:

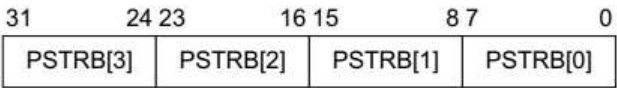


Fig.5: Byte Lane Mapping

The write strobe signals, ‘PSTRB’, enable sparse data transfer by indicating valid byte lanes on the write data bus. Each ‘PSTRB’ corresponds to one byte, with ‘PSTRB[n]’ linked to ‘PWDATA[(8n+7):(8n)]’. This relationship is shown on a 32-bit data bus in above Fig. For read transfers, the bus master must drive all bits of PSTRB LOW.

APB read transfers:

With no wait state:

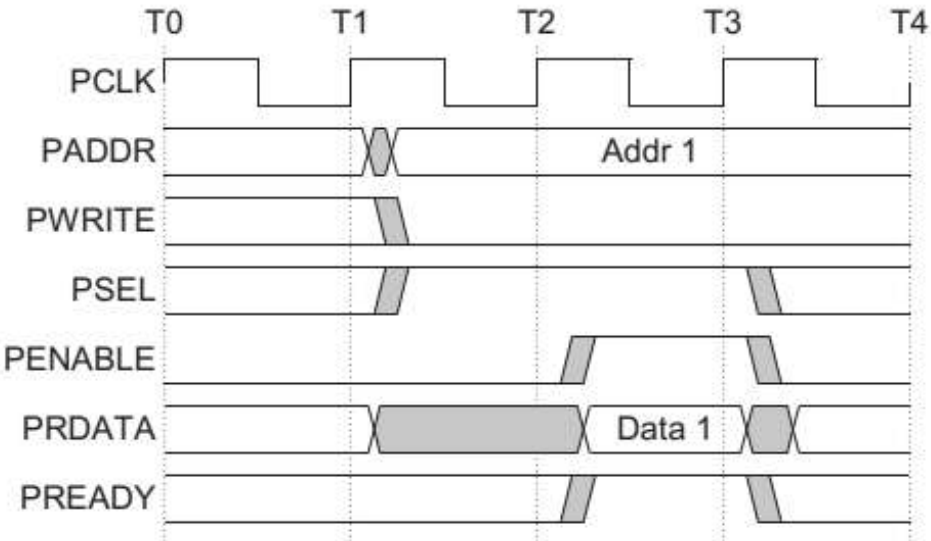


Fig.6: Read Transfer with no wait state

The above figure shows the read transfer with no wait state, and the condition is that the slave must provide the data before the end of the read transfer.

With wait state:

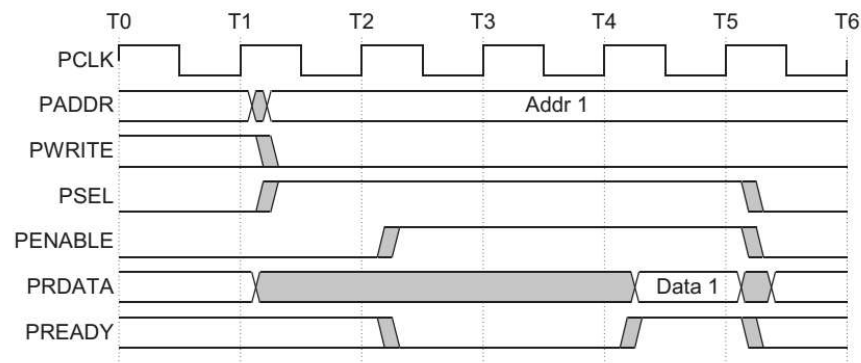


Fig.7: Read Transfer with wait state

Error response:

Write transfer:

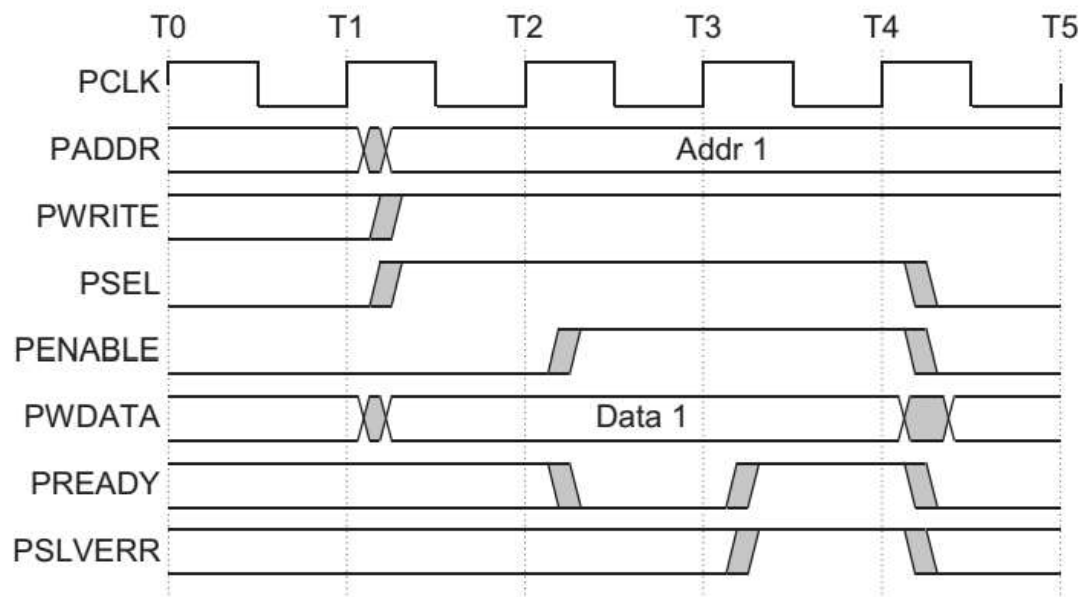


Fig.8: Example failing write transfer

The above figure shows that the write transfer completed with an error response indicated by the PSLVERR signal.

Read transfer: A read transfer can also be completed with an error response, indicating no valid read data is available.

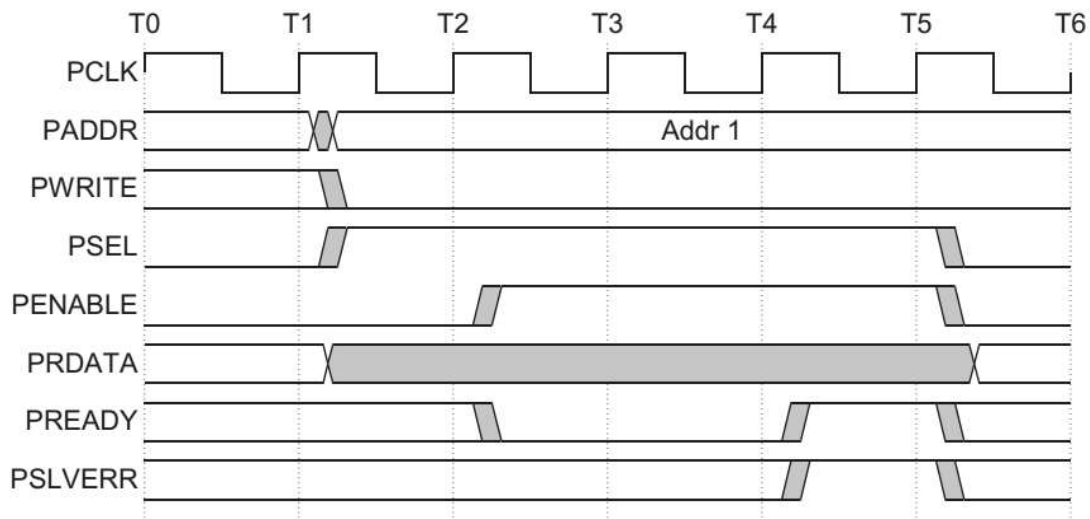


Fig.9: Example failing read transfer

Protection unit support

APB 4 has PPROT[2:0] signals

PPROT[2:0]	Protection Level	Description
[0]	1= privileged access 0= normal access	Normal or privileged, PPROT[0] • LOW indicates a normal access • HIGH indicates privileged access.
[1]	1= non secure access 0= secure access	Secure or non-secure, PPROT[1] • LOW indicates a secure access • HIGH indicates a non-secure access.
[2]	1= instruction access 0= data access	Data or Instruction, PPROT[2] • LOW indicates a data access • HIGH indicates instruction access.

Operating States:

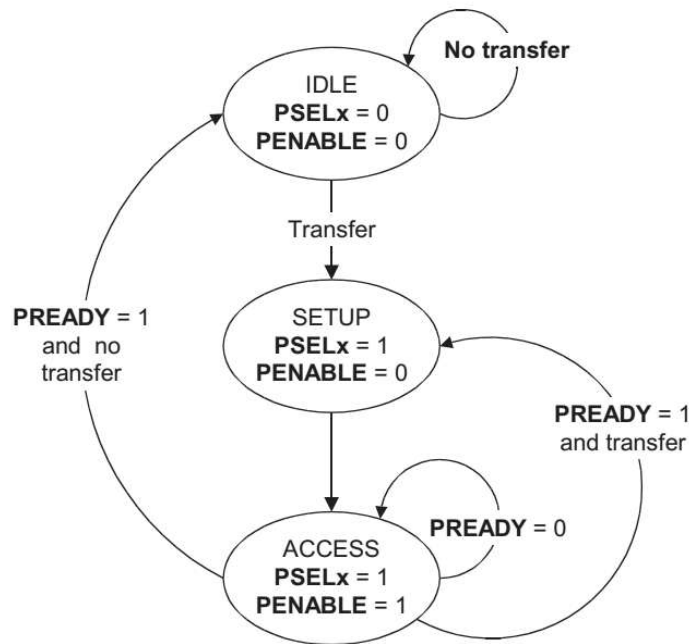


Fig.10: State diagram of APB operation

The state machine operates through the following states:

- **IDLE:** This is the default state of the APB.
- **SETUP:** When a transfer is required, the bus moves into the SETUP state, where the appropriate select signal, PSELx, is asserted. The bus remains in the SETUP state for one clock cycle and always moves to the ACCESS state on the next rising edge of the clock.
- **ACCESS:** The enable signal, PENABLE, is asserted in the ACCESS state. The address, write, select, and write data signals must remain stable during the transition from the SETUP to ACCESS state.

The **PREADY** signal from the slave controls exit from the ACCESS state:

- If **PREADY** is held LOW by the slave, then the peripheral bus remains in the ACCESS state.
- If **PREADY** is driven HIGH by the slave, then the ACCESS state is exited, and the bus returns to the IDLE state if no more transfers are required. Alternatively, the bus moves directly to the SETUP state if another transfer follows.

Advantages of APB

- Minimal power consumption
- Reduced interface complexity
- APB protocol is not pipelined; it is used to connect to low-bandwidth peripherals that do not require the high performance of the AXI protocol.
- Integrating APB peripherals into any design flow is more straightforward due to the rising edge of the clock transition.

Advance Trace Bus

Introduction

The ATB protocol, part of the AMBA family, facilitates trace data transfer in CoreSight systems. Introduced in AMBA 3 (ATBv1.0 or Issue A), AMBA 4 (ATBv1.1 or Issue B), and AMBA 5 (ATB-C or Issue C), it provides a common bus for format-independent trace data exchange. Components with trace capabilities use ATB interfaces as Transmitters (data generation) or Receivers (data reception).

Key features include data stalling with valid-ready signals, byte-valid cycle control, origin identification via associated IDs, and support for diverse trace protocols and formats. This enables seamless integration and management of trace data across components in advanced trace systems.

The ATB protocol includes several trace components, each playing a specific role:

Trace Sources

1. **High-performance Trace Macrocell (HTM):** Outputs trace data describing the Advanced High-performance Bus (AHB) interconnect being monitored.
2. **Trace Macrocells (xTM):** Generate trace data from connected processors. Examples include:
 - **Embedded Trace Macrocell (ETM):** Produces instruction and optional data traces.
 - **Program-flow Trace Macrocell (PTM):** Outputs instruction trace.

Trace Links

1. **Trace Funnel:** Combines multiple trace sources into a single bus stream.
2. **Trace Replicator:** Splits one ATB receiver interface into two independent transmitters.

Trace Sinks

1. **Embedded Trace Buffer (ETB):** On-chip storage for trace data.
2. **Trace Port Interface Unit (TPIU):** Sends trace data to an external trace port for storage or analysis.

System Operations

The ATB system ensures reliable data transfer by propagating flush requests from sinks to sources. Trace links pass these requests and confirm when all data is received. This ensures complete trace data transfer and system integrity.

Evolution of ATB

Issue A (AMBA3)	Issue B (AMBA4)	Issue C (AMBA5)
• ATB was introduced with trace transfer using a ready-valid handshake mechanism for wait response. • Included flush functionality to ensure trace data consistency	• Added SYNCREQ signal for synchronization requests. • Updated reserved ATID values and interface signaling rules. • Introduced trace synchronization signaling and synchronization request signaling. • Made ATCLKEN an optional signal. • Removed the recommendation for ATDATA to be at least 32 bits wide.	• Updated terminology throughout the specification. • Introduced wake-up signaling

ATB Signals

Global Signals

Signal	Transmitter	Receiver	Description
ATCLK	Input	Input	Global ATB clock.
ATCLKEN	Input	Input	Enable signal for ATCLK domain. From ATB Issue B, this signal is optional.
ATRESETn	Input	Input	ATB interface reset, active LOW. This signal is asserted LOW asynchronously, and deasserted HIGH synchronously

Data Signal

Signal	Transmitter	Receiver	Clamp Value	Description
ATBYTES[m:0]	Output	Input	LOW	The number of bytes on ATDATA to be captured, minus 1
ATDATA[n:0]	Output	Input	LOW	Trace data

ATID[6:0]	Output	Input	LOW	An ID that uniquely identifies the source of the trace
ATREADY	Input	Output	HIGH	The Receiver is ready to accept data
ATVALID	Output	Input	LOW	A transfer is valid during this cycle. If LOW, all other AT signals must be ignored

Flush Control Signal

Signal	Transmitter	Receiver	Clamp Value	Description
AFVALID	Input	Output	LOW	The flush signal to indicate that all buffers must be flushed because trace capture is about to stop
AFREADY	Output	Input	HIGH	The flush acknowledge to indicate that buffers have been flushed

Synchronization request signal

Signal	Transmitter	Receiver	Clamp Value	Description
SYNCREQ	Input	Output	LOW	The synchronization request signal to request the insertion of synchronization information in the trace stream

Wake-up signal

Signal	Transmitter	Receiver	Description
ATWAKEUP	Input	Output	The wake-up signal indicating any activity associated with an ATB interface

ATB Connection

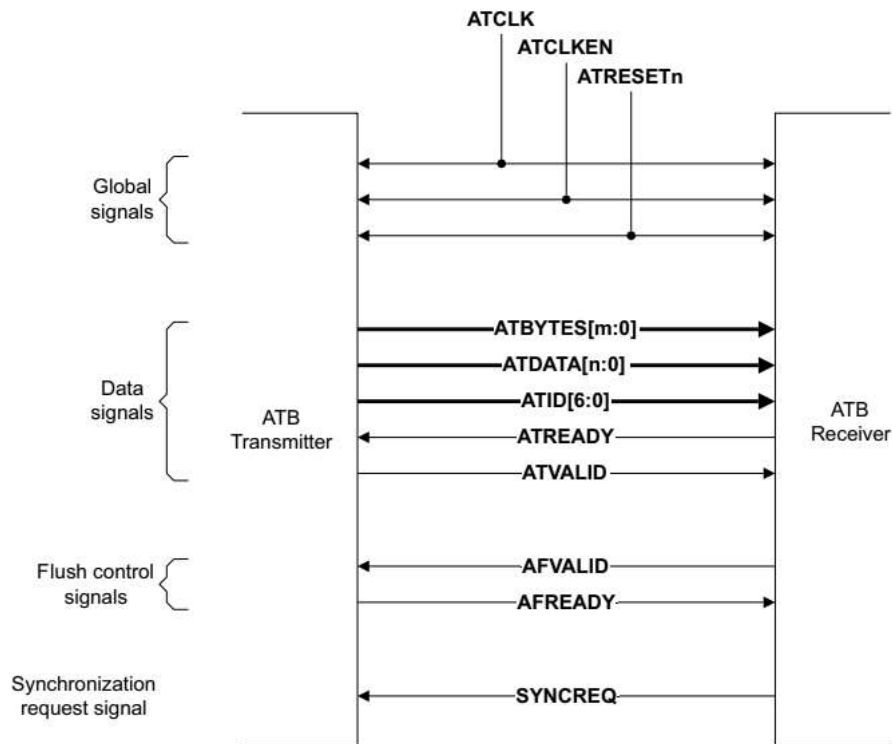


Fig. 11: ATB signaling between Transmitter and Receiver

ATB Flow Control

- The source generates the $ATVALID$ signal to indicate when the data or control information is available.
- The destination generates the $ATREADY$ signal to indicate that it accepts the data or control information.
- Data transfer occurs only when both the $ATVALID$ and $ATREADY$ signals are HIGH.

Trace Transfer

- Transmitter generate $VALID$ signal for the trace signal
- In response, the receiver can generate a $READY$ signal. It can lower the $READY$ to make the transmitter resend the trace data.
- If the receiver is not ready to receive the trace, it will not generate the $READY$ signal. The Transmitter will send the same until the $READY$ signal is generated for that trace.
- Source can only send $VALID$ signal after the system is fully stable a reset

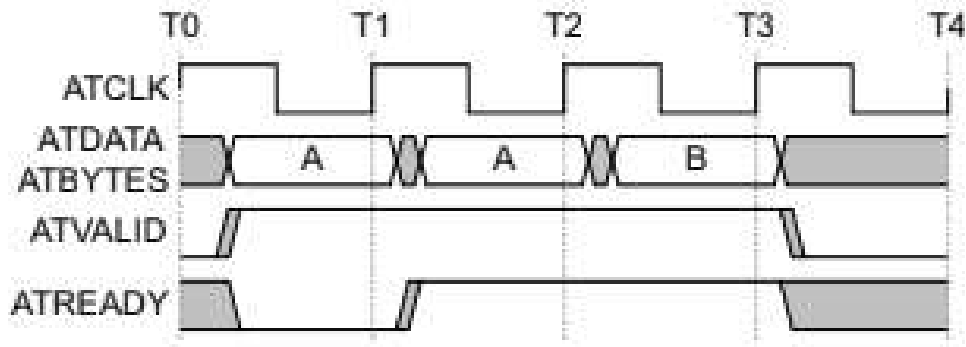


Fig.12: Timing diagram of trace transfer

ATB Features

- **ATIDs :** Every trace data has unique identifier

Use of ATIDs is as follows:

- ❖ Differentiate trace data from various sources.
- ❖ Identify high-bandwidth and low-bandwidth trace components for selective filtering downstream.
- ❖ Align synchronization by changing the ID at key points, like the start of a trace packet.

Rules related to ATIDs:

- ❖ Most trace source use a single,static ATID
- ❖ Since Issue B, some ATIDs are reserved for trace triggering and cortex processors
- ❖ ATIDs either can be fixed during design or could be programmable with help of a debugger.
- ❖ ATIDs are captured at the same time as ATDATA
- ❖ Order of trace should be same even if they different ATIDs (different source)

• Flush:

Initiation of Flush:

- ❖ A buffer flush occurs to remove any remaining trace data from FIFOs or buffers to prepare for new data.
- ❖ Flush is typically triggered when the system is powered down, the clock is stopped, or a trace capture device is about to stop capturing data.

AFVALID and AFREADY Signals:

- ❖ **AFVALID (Flush Valid):** Indicates that a flush is in progress and the trace data in the FIFO must be output.

- ❖ **AFREADY (Flush Ready):** Indicates that all trace data generated at the time AFVALID was asserted has been output.
- ❖ **Sequence:**
 - AFVALID is asserted by the Transmitter to initiate the flush.
 - AFREADY is asserted by the Transmitter one cycle after all required data is output.

Handling of New Trace Data:

- ❖ Any trace data generated after the flush is stored in the FIFO and output later.
- ❖ AFVALID remains asserted until AFREADY is deasserted.

Priority During Flush:

- ❖ During a flush, trace funnels give the highest priority to sources that have not yet asserted AFREADY.

Non-cancelable Flush:

- ❖ Once a flush is initiated, it cannot be canceled by the Receiver interface.

Clock Synchronization:

- ❖ ATB signals, including AFVALID and AFREADY, must be sampled on the rising edge of ATCLK.

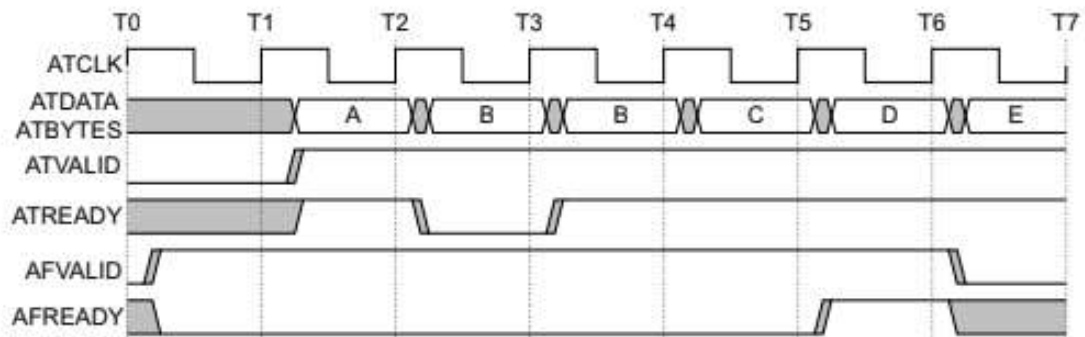


Fig. 13: Flush Signalling

Key Points in ATB Flush:

- ❖ **Triggering Events:**
 - System power down or clock stop.
 - Trace capture device stopping data capture.
 - Periodic flush to ensure complete trace capture.
- ❖ **Flush Signals:**
 - AFVALID indicates flush initiation.
 - AFREADY indicates flush completion for the current data.
- ❖ **Trace Data Handling:**
 - Trace generated after AFVALID is stored in FIFO and processed later.
 - Flush ensures no trace is lost during critical events.
- ❖ **Protocol Rules:**
 - Flush cannot be canceled after initiation.
 - AFREADY must be asserted exactly one cycle after flushing the required data.
- ❖ **Periodic Flushes:**
 - Used in scenarios with uncertain trigger timing to guarantee complete trace capture.

● Signalling a Trace Trigger

A signal from a trace source to a trace sink, indicating a significant condition or event. Used to prepare for stopping trace capture.

How is a Trace Trigger Signaled?

ATID: Must be 0x7D to indicate a trace trigger.

ATBYTES: Shows the number of trace triggers in this cycle:

- ❖ 0x0: Only one trace trigger is generated (recommended).
- ❖ 0x1: Two trace triggers are generated.

ATDATA: Indicates the source of the trace trigger:

- ❖ Each byte represents a separate trigger
- ❖ A non-zero byte indicates the ID of the trace source that generated the trigger.
- ❖ A zero byte means the source is unknown (not recommended).

● Wakeup Signalling

- ❖ **Purpose:** Indicates activity on the ATB interface.
 - Used for glitch-free signals to control power and clocks.
- ❖ **Property:** **Wakeup_Signal** specifies whether a wake-up signal is supported.
 - True: Wake-up signal is present.
 - False: Wake-up signal is not present (default if not declared).
- ❖ **Restriction:** Only for ATB Issue C interfaces.

ATWAKEUP Rules:

- ❖ Synchronization:
 - Synchronous to ATCLK, suitable for crossing clock domains.
- ❖ Glitch-Free:
 - Must be generated from a register.
- ❖ Assertion Timing:
 - Can be asserted anytime before/after ATVALID.
 - Must remain HIGH if asserted alongside ATVALID until ATREADY is asserted.
- ❖ Receiver Behaviour:
 - Receiver may wait for ATWAKEUP before asserting ATREADY.
 - Potential deadlock if ATWAKEUP is never asserted.

Recommendations:

- ❖ Assert ATWAKEUP at least one cycle before ATVALID to avoid delays.
- ❖ Deassert ATWAKEUP when no further transfers are needed.
- ❖ Avoid asserting and deasserting ATWAKEUP without a transaction.

Signalling rules:

Sampling Rules:

- ❖ ATB signals must be sampled on the rising edge of ATCLK.
- ❖ If ATREADY is LOW and ATVALID is HIGH in a cycle, ATVALID, ATDATA, ATID, and ATBYTES must retain their values in the next cycle.
- ❖ If ATVALID is LOW, ignore ATREADY.

Data and Byte Handling:

- ❖ If both ATREADY and ATVALID are HIGH, capture the bottom bytes of ATDATA, aligned to the least significant byte.
- ❖ Zero bytes cannot be captured.

ATDATA and ATBYTES Relationship:

- ❖ The width of ATDATA (in bits) must be a power of two, at least 8 bits.
- ❖ The relationship between ATBYTES and ATDATA widths is defined by:

$$m = \log_2(n+1) - 4$$

where m is the width of ATBYTES and n is the width of ATDATA.

Trace Order and ID Rules:

- ❖ Capture ATDATA and ATID bytes simultaneously.
- ❖ Preserve the order of trace bytes, even between traces with different ATIDs.
- ❖ When trace data is funneled into a single bus, the order cannot be changed.

AFVALID and AFREADY Rules:

- ❖ When AFVALID is asserted, it must remain HIGH until AFREADY is HIGH.
- ❖ If AFVALID is asserted, the transmitter must output buffered trace data immediately.
- ❖ After AFREADY is asserted, AFVALID must be deasserted in the next cycle unless another flush is needed.

Reset Rules:

- ❖ ATRESETn can be asserted asynchronously but must be deasserted synchronously on a rising edge of ATCLK.
- ❖ ATRESETn is active LOW. Trace sources can drive ATVALID HIGH only after ATRESETn is HIGH at a rising edge of ATCLK.

Receiver and Transmitter Behavior:

- ❖ If a receiver cannot respond, it must drive ATREADY HIGH and AFVALID LOW.
- ❖ If a transmitter cannot respond, it must drive AFREADY HIGH and ATVALID LOW.

ATID Reserved Values:

- ❖ ATID values 0x00, 0x70-0x7C, 0x7E, and 0x7F are reserved and must not be used.

Unused Signals:

- ❖ ATID, ATBYTES, and ATDATA can take any value for unused inputs.

Advanced High-Performance Bus (AHB)

Introduction:

The AHB protocol was introduced in AMBA 2 and replaces the APB protocol specifically for high-performance, high-clock frequency systems. As other AMBA protocols, the AXI protocol carries a master-slave protocol and is used as the backbone bus for high-performance systems. AHB achieves high performance through features like:

- Burst transfers
- Split transactions
- Single-cycle master handover
- Single-clock edge operation
- Non-tristate implementation
- Wider data buses (64/128 bits).

AHB interface is used for bus masters, on-chip memory blocks, external memory interfaces, high-bandwidth peripherals with FIFO interfaces, and DMA slave peripherals.

The objective of AHB:

AHB interface is used for bus masters, on-chip memory blocks, external memory interfaces, high-bandwidth peripherals with FIFO interfaces, and DMA slave peripherals.

Evolution of AHB:

The AHB specification, first introduced in AMBA 2, evolved through the following key versions:

- AMBA 2 AHB: first release, using 'Bus Request' and 'Bus Granted' handshaking signals for master arbitration.
- Multi-layer AHB: Introduced with the AHB Bus Matrix in the AMBA Design Kit, enabling concurrent transfers in multi-master systems without handshaking signals (unofficially called "AHB Lite").
- AMBA 3 AHB Lite: Officially named in AMBA 3, this version removed handshaking signals and simplified the protocol for Arm systems.
- AMBA 5 AHB (AHB5): Latest release with support for TrustZone in Armv8-M, exclusive access sideband signals, cache attributes, and backward compatibility with AHB Lite.

AHB5 builds on previous versions, allowing reuse of existing AHB Lite bus slaves.

AHB-based microcontroller:

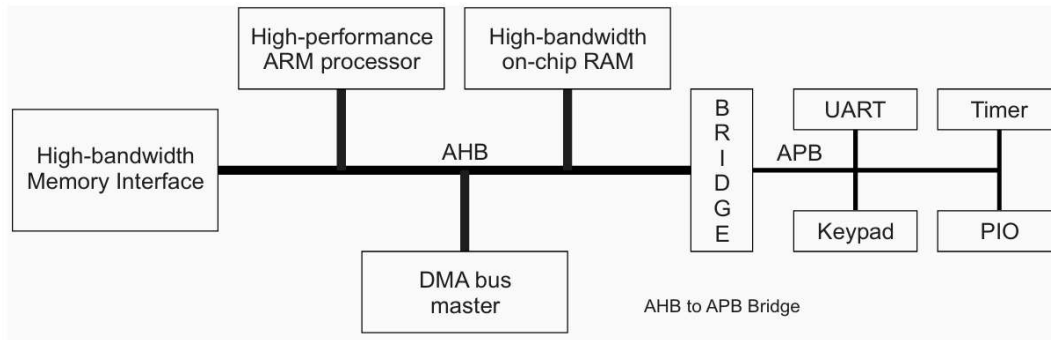


Fig.14: AHB-based microcontroller Diagram

An AMBA-based microcontroller includes a high-performance AHB backbone bus for the CPU and DMA devices, sustaining external memory bandwidth. It connects to a narrower APB bus via a bridge, which links the processor to peripherals and AHB to APB. AHB supports multiple masters and slaves, enabling efficient communication.

AHB signals:

H is the signal prefix that indicates an AHB signal.

Signal	Source & Destination	Description
HCLK	Clock Source → All AHB blocks	Clock signal
HRESETn	Reset Source → All AHB blocks	Active-low reset signal
HSEL	Address decoder → Slave	Device select
HADDR[31:0]	Master → Slave	Address bus
HTRANS[1:0]	Master → Slave	Transfer control
HWRITE	Master → Slave	Write control (1=Write, 0=Read)
HSIZE[2:0]	Master → Slave	Size of Transfer
HBURST[2:0]	Master → Slave	Burst type of Transfer
HPROT[3:0]/[6:0]	Master → Slave	Transfer Protection control (4 bits in AHB Lite, 7 bits in AHB5)
HWDATA[31:0]	Master → Slave	Write data
HRDATA[31:0]	Master ← Slave	Read Data

HRESP[1:0]	Master ← Slave	Slave response (2 bit wide in AMBA 2, 1 bit wide in AHB Lite, and AHB5)
HREADY	Master ← Slave	Slave ready (indicates transfer completed)

Bus interconnection:

The AMBA AHB protocol uses a central multiplexor interconnection scheme.

- **Bus master:** The Master provides address and control signals and initiates the transaction.
- **Arbiter:** prioritize master's address and control signals.
- **Decoder:** Controls the read data and response signal multiplexor by selecting the appropriate signals from the slave.

This design enables efficient communication between multiple masters and slaves. The figure below illustrates an AMBA AHB system with three masters, four slaves, an arbiter, a decoder, an address and control multiplexor, and read and write data multiplexors.

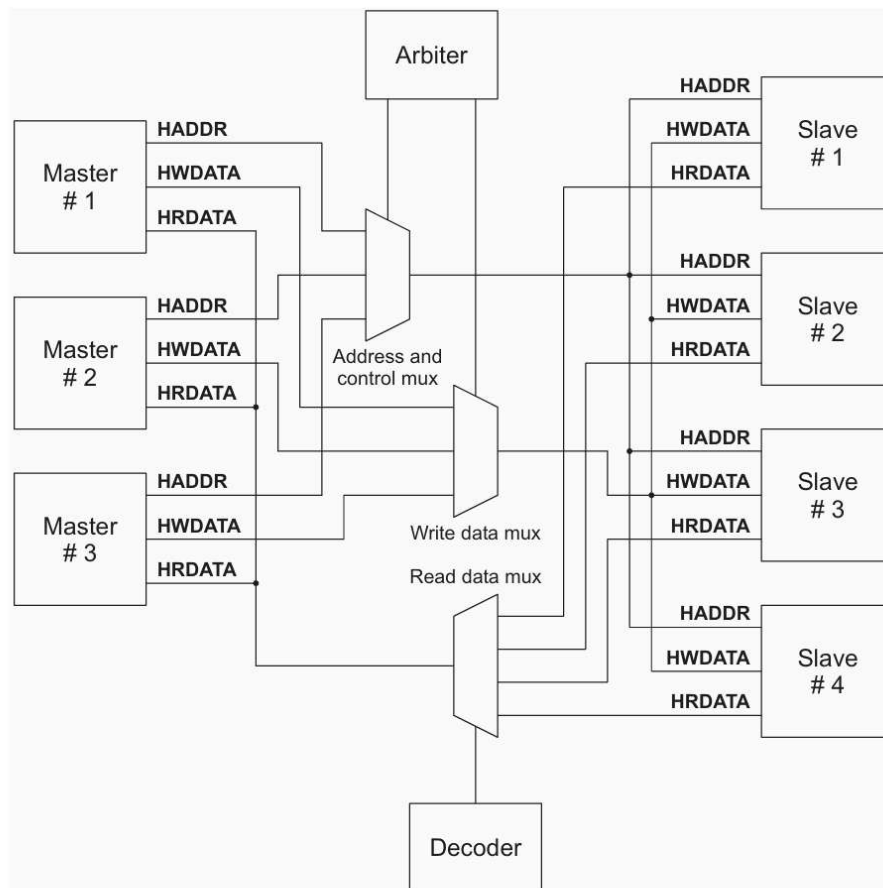


Fig.15: Multiplexor interconnection

Basic Transfer:

An AHB transfer consists of two phases:

- Address phase: Lasts a single cycle.
- Data phase: This is managed using the HREADY signal, which may take multiple cycles.

With no wait state:

In a simple transfer with no wait states:

- The master drives the address and control signals onto the bus after the rising edge of HCLK.
- The slave samples these signals on the next rising edge of the clock.
- The slave begins driving the response, which is sampled by the master on the third rising edge of the clock.

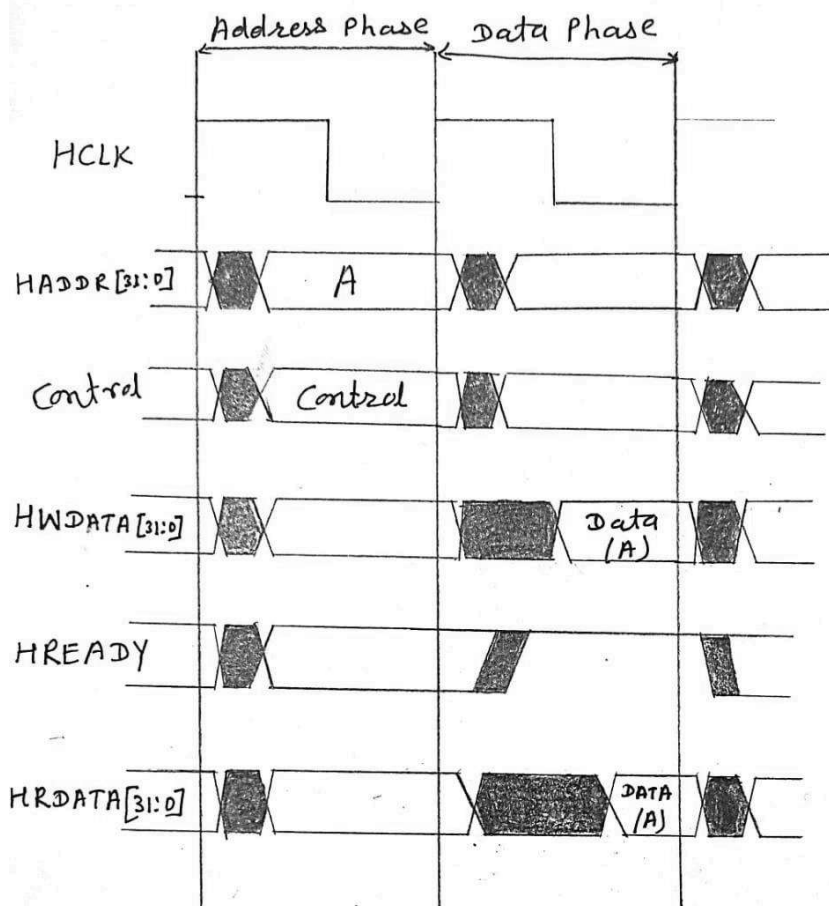
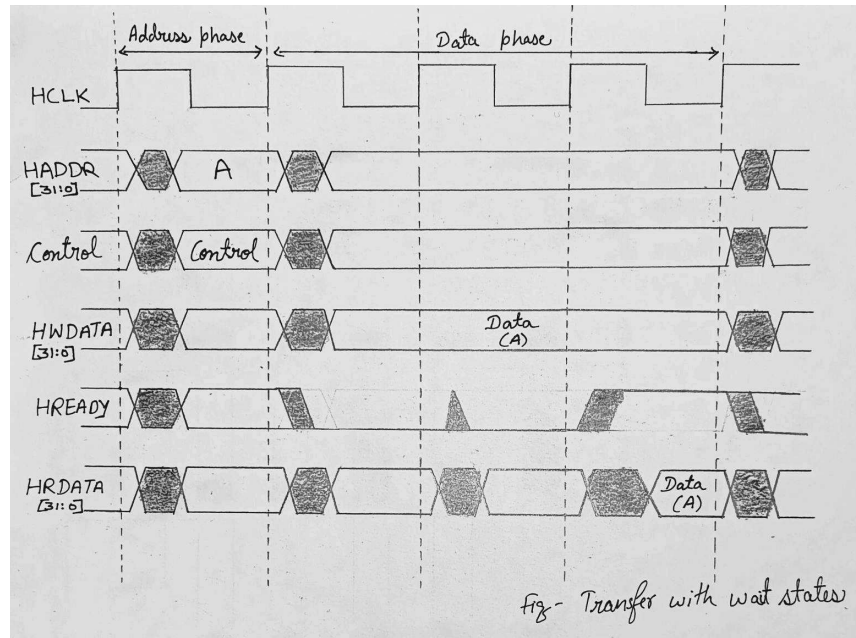


Fig :- Simple Transfer in AHB Protocol

With wait state:

When a slave inserts wait states, the data phase is extended, providing more time for transfer completion:

- For write operations, the master holds the data stable throughout the extended cycles.
- For read transfers, the slave doesn't need to provide valid data until near the end of the transfer.



Multiple Transfers:

In the case of multiple transfers, the address phase of the current transfer can overlap with the extended data phase of the previous transfer. This is fundamental to pipelining:

- When a transfer is extended with wait states, it extends the address phase of the following transfer.
- This behavior enables higher performance as the bus is kept active.
- The figure below demonstrates three transfers to unrelated addresses (A, B, and C), showing this concept.

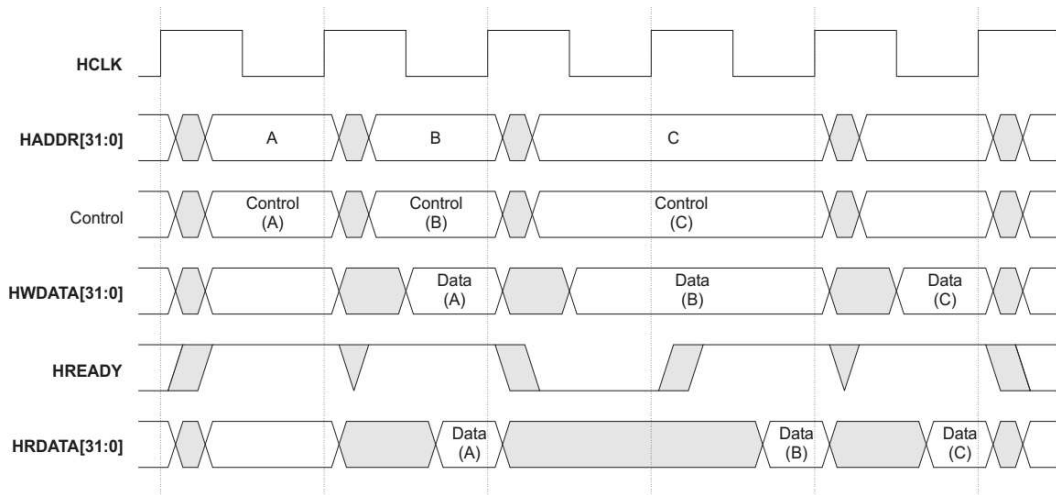


Fig.18: Multiple transfers

Transfer type:

Every transfer in the AHB protocol can be classified into one of four types, indicated by the HTRANS[1:0] signals, as shown in the table below.

HTRANS[1:0]	Type	Description
00	IDLE	No data is required.
01	BUSY	The bus master is continuing with a burst of transfers, but the next transfer cannot take place immediately.
10	NONSEQ	It indicates the first transfer of a burst or a single transfer.
11	SEQ	The remaining transfers in a burst are sequential.

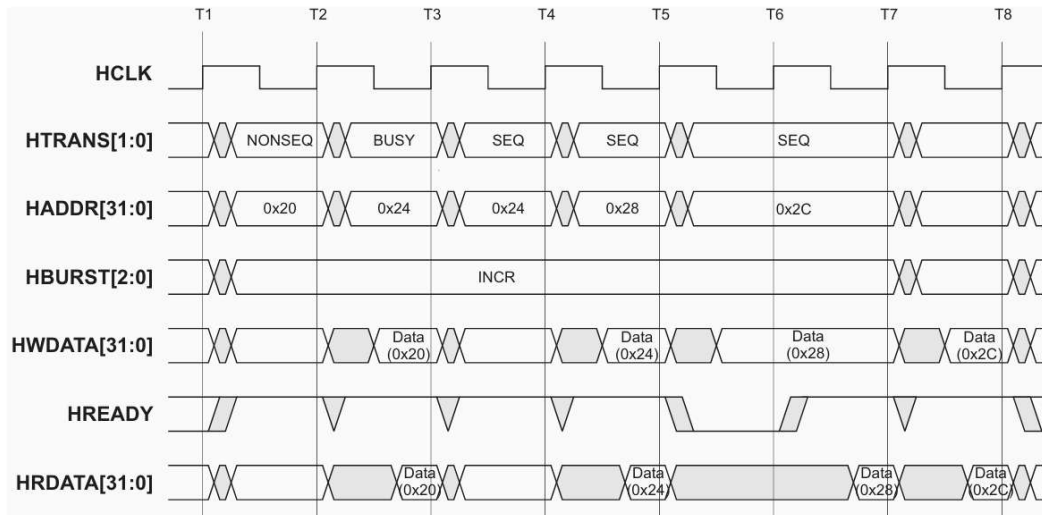


Fig.19: Transfer type examples

- The first transfer is the start of a burst and, therefore, is NONSEQUENTIAL.
- The second transfer is delayed using a BUSY transfer to give the master time before the next transfer starts.
- The third transfer is SEQUENTIAL, with the slave inserting a single wait state (HREADY) as it completes the transfer.
- The final transfer of the burst is complete with zero wait states.

This classification ensures that the AHB protocol can handle various types of transfers efficiently, depending on the needs of the system and the data flow between masters and slaves.

Burst operation:

AMBA AHB supports four, eight, and sixteen-beat bursts, as well as undefined-length bursts and single transfer.

HBURST[2:0]	Type	Description
000	SINGLE	Single transfer
001	INCR	Incrementing burst of unspecified length
010	WRAP4	4-beat wrapping burst
011	INCR4	4-beat incrementing burst
100	WRAP8	8-beat wrapping burst
101	INCR8	8-beat incrementing burst
110	WRAP16	16-beat wrapping burst

111	INCR16	16-beat incrementing burst
-----	--------	----------------------------

Bursts must not cross a 1KB address boundary, so masters should not start a fixed-length incrementing burst that would cross this boundary.

The total data transferred in a burst is calculated by multiplying the number of beats by the data size of each beat, as indicated by the HSIZE[2:0] signal.

Wrapping Burst:

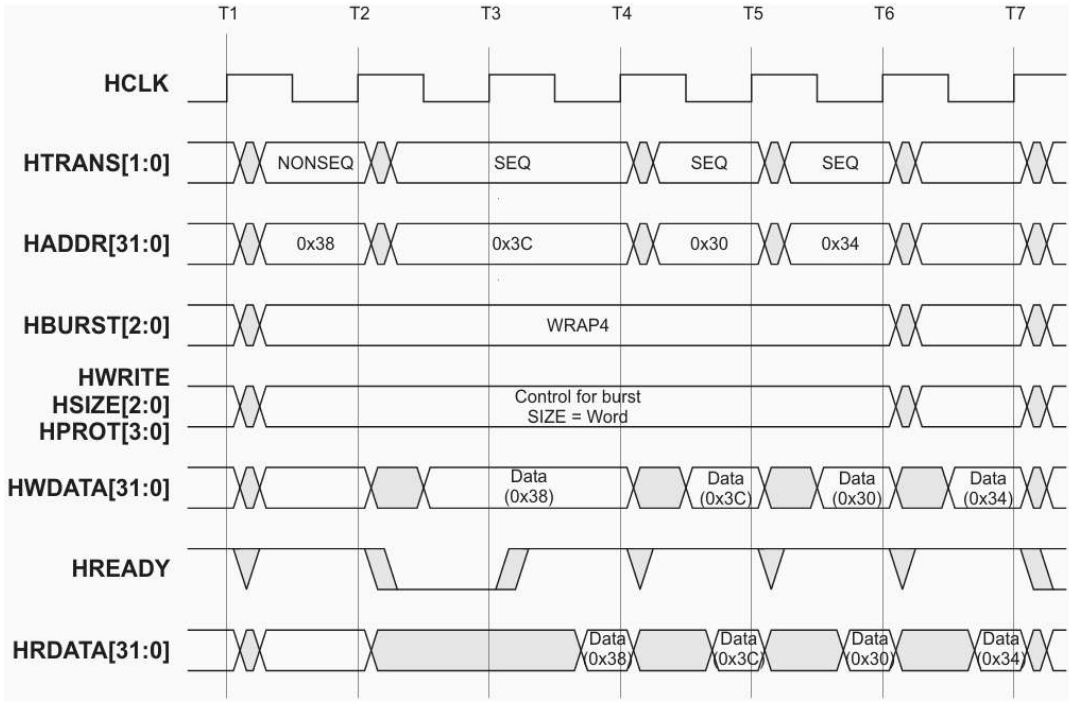


Fig.20: Four-beat wrapping burst

In a wrapping burst, addresses wrap at the boundary (e.g., 16-byte for a 4-beat word burst). If the starting address is 0x34, the transfers are 0x34, 0x38, 0x3C, and 0x30.

Incremental burst:

In an incrementing burst, addresses increment sequentially with no wrapping. The burst continues beyond the defined boundary.

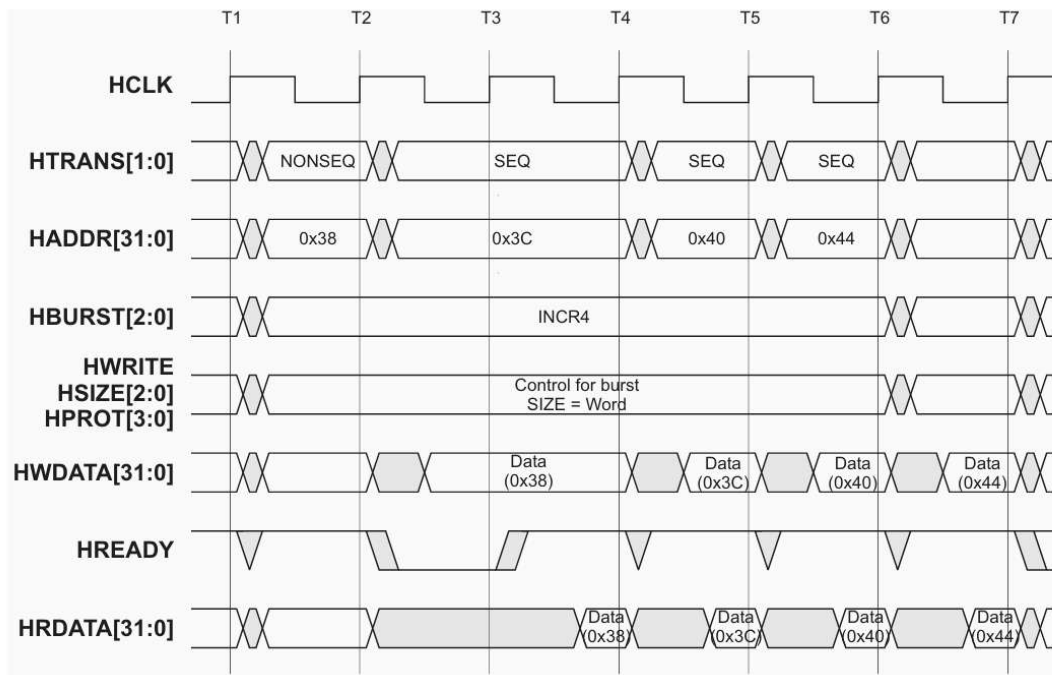


Fig.21: Four-beat incremental burst

Undefined-length Bursts:

Undefined-length bursts allow transfers with an unspecified number of beats, which is useful when a master cannot complete a burst in a single operation.

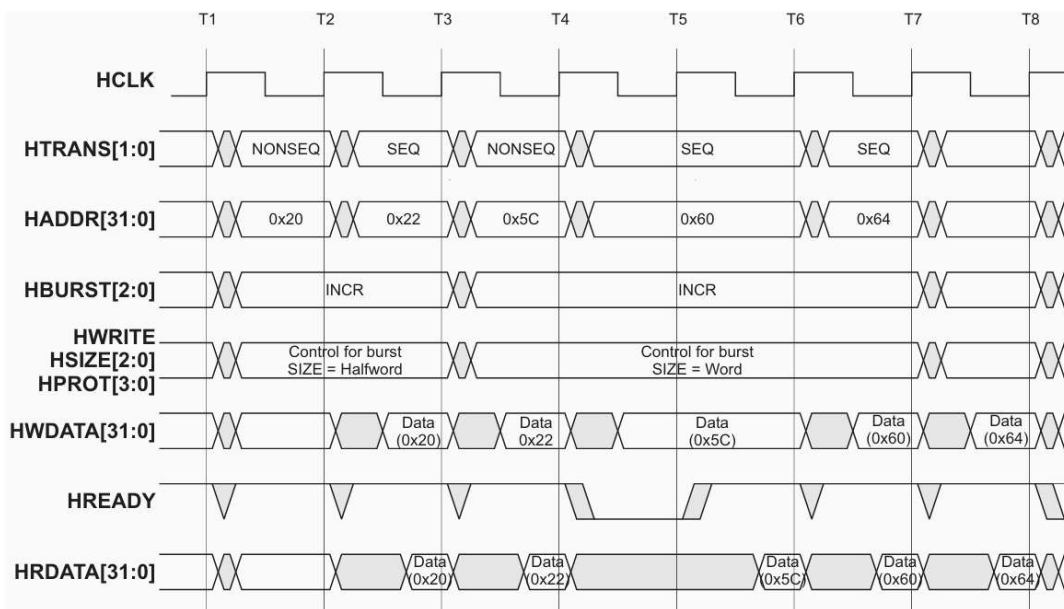


Fig.22: Undefined-length bursts

The above figure shows two bursts:

- Two halfword transfers starting at address 0x20. The halfword transfer addresses increment by 2.
- Three word transfers starting at address 0x5C. The word transfer addresses increment by 4.

Address Decoding:

The figure below shows the working of the address decoder.

- The arbiter first selects which master can use the bus on a priority basis. Then, that particular master provided the address by HADDR_Mx signal, which was further transferred to slaves.
- A central address decoder generates the select signal (HSELx) for each slave. The transfer will complete only when the HREADY signal is HIGH.

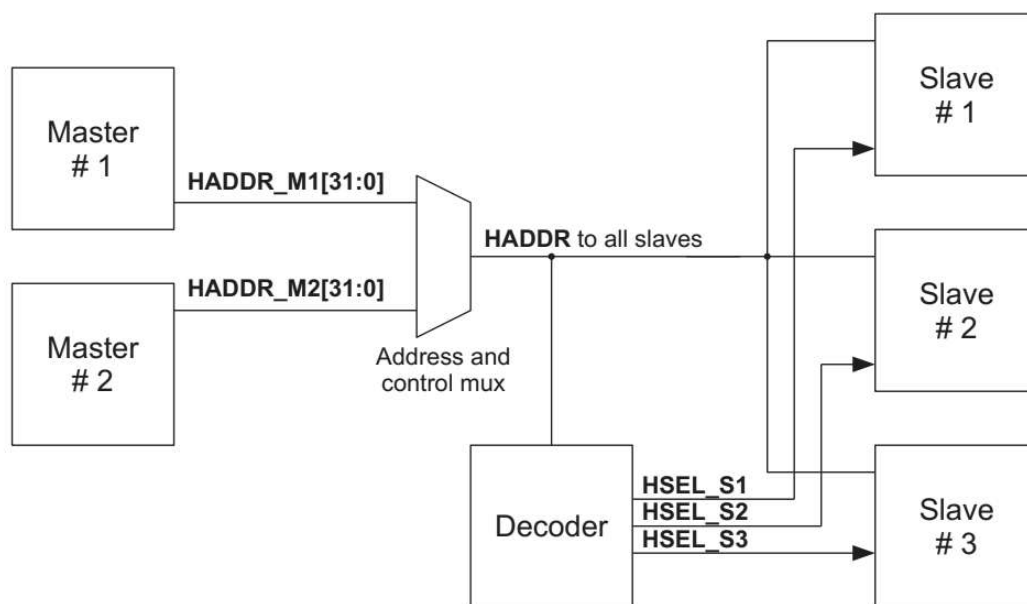


Fig.23: Slave select signals

Transfer Response:

Slaves use the HREADY signal to insert wait states and indicate the transfer's status with the HRESP[1:0] response signals:

HRESP[1:0]	Response	Description
00	OKAY	Indicates successful transfer completion
01	ERROR	Indicates a transfer error
10	RETRY	Indicates the transfer is incomplete, and the master should retry until

		completion.
11	SPLIT	Indicates the transfer is incomplete, but the slave will later request bus access for the master to complete the transfer

Slaves typically use **RETRY** or **SPLIT** for high-latency operations, freeing the bus for other masters. When deciding the final response after wait states, the slave must first drive OKAY.

Transfer with Retry Response:

In AMBA-AHB, a RETRY response signals the master to retry a failed transfer until it is completed. Two cycles are required for ERROR, SPLIT, or RETRY responses to allow the master to cancel the next transfer and switch to IDLE.

The figure below shows:

- First the master initiates a transfer at address A (HTRANS = NONSEQ) then in the second clock cycle i.e. in T2, the master sets the next address to A + 4 (HTRANS = SEQ).
- The slave at A cannot complete the transfer and issues RETRY (HREADY LOW) and therefore the master cancels the A + 4 transfer, switching to IDLE in T3.
- In T4 i.e. fourth clock cycle, the slave completes the transfer at A with indicating OKAY HRESP in T5 (HREADY HIGH, HRESP = 00).

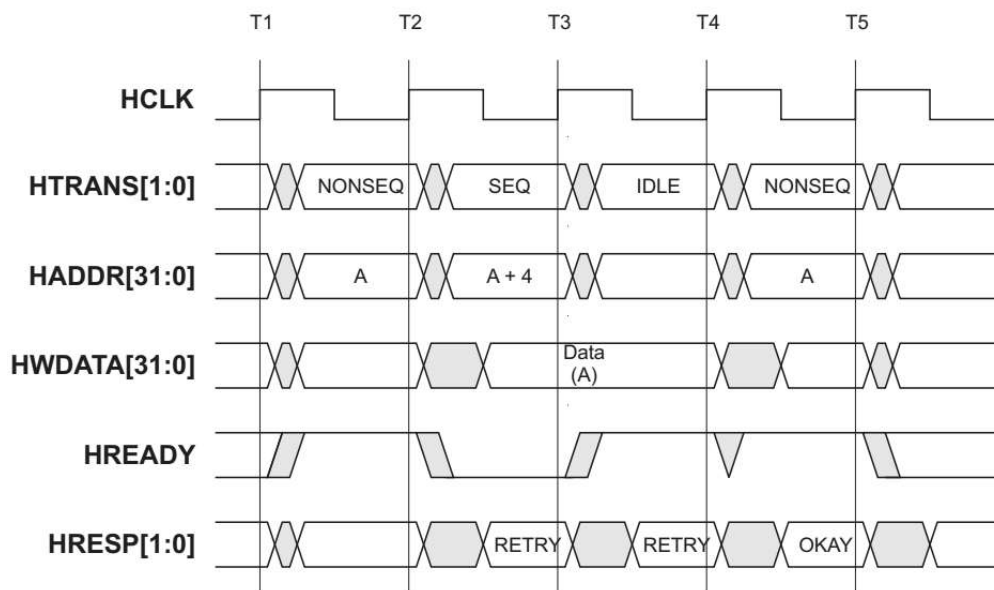


Fig.24: Transfer with retry response

Error Response:

The transfer is unsuccessful if an ERROR response is returned. The slave sends the error information to the master, and the master may decide to abort or continue further transfers.

The figure below illustrates:

- The transfer would be initiated at address A by the master with control and data signals, and during the second cycle, the slave holds HRESP = OKAY.
- The slave returns an ERROR response during the next two cycles, implying its unsuccessful: HRESP = ERROR. Then, based on the master, the burst can be terminated.

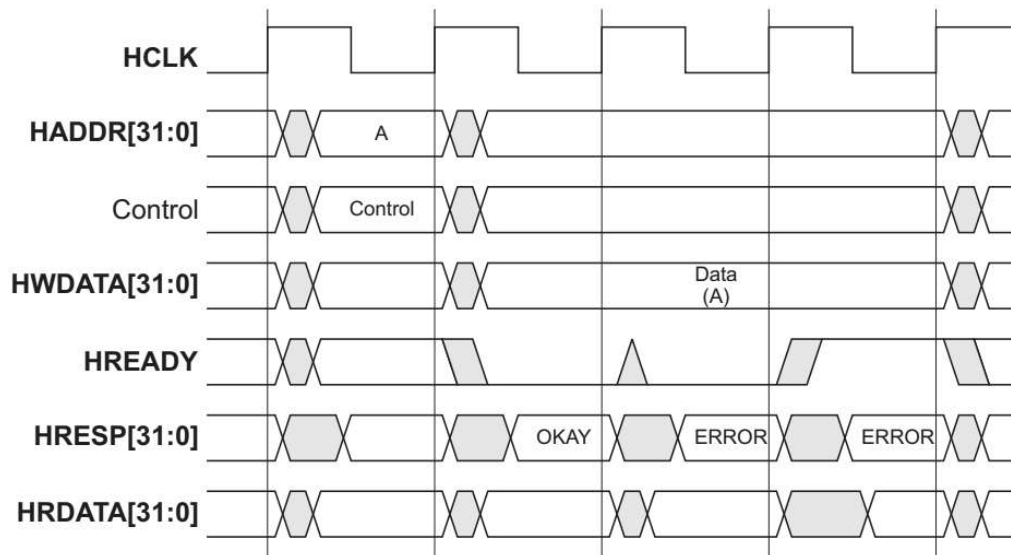


Fig.25: Error response

Advantages of AHB:

AMBA-AHB protocol has several advantages over earlier AMBA protocols such as ASB (Advanced System Bus):

- 1. High Performance: Pipelined data transfer is supported, allowing for high-speed communication between master and slave.
- 2. Burst Transfers: Multiple data transfers are handled efficiently, reducing overhead and improving bandwidth.
- 3. Error Handling: There are clear mechanisms (e.g., 'HRESP') for handling errors in data transfers, ensuring reliable operation.
- 4. Scalability: Good for complex SoC designs having multiple masters and slaves.
- 5. Ease of Use: Simplified arbitration and burst operations reduce design complexity.

Advanced High - performance Bus Lite (AHB lite)

The AHB Lite protocol is a simpler version of the full Advanced High-Performance Bus (AHB) protocol, designed for systems with only one master. By supporting only a single master, AHB Lite avoids the complexities associated with multiple masters and introduces the following simplifications:

Simplifications in AHB Lite:

1. **No Arbitration:**

Since there's just one master, there's no competition for bus access. This eliminates the need for arbitration logic to decide which master controls the bus.

2. **No HBUSREQ or HGRANT Signals:**

In a full AHB setup, masters use the HBUSREQ (Bus Request) signal to request bus access and receive the HGRANT (Grant) signal when access is granted. AHB Lite doesn't need these signals, as the single master always has access to the bus.

3. **No HSEL Signal:**

The HSEL (Slave Select) signal, used to select which slave device is active in a full AHB system, isn't necessary for AHB Lite. The master communicates directly with the intended slave.

4. **Simplified HRESP Signal:**

The HRESP signal, which indicates the status of a transaction, is reduced to a single bit in AHB Lite. Complex responses like "split" or "retry" (present in the full AHB) are not needed.

5. **No Early Bus Termination:**

Transactions in AHB Lite proceed without being prematurely stopped or terminated, simplifying the protocol.

Connecting Full AHB and AHB Lite Components:

When connecting a full AHB master to an AHB Lite slave:

- Leave the HBUSREQ signal unconnected (it's unused).
- Tie the HGRANT signal to logic high (1) so the master always has bus access.

When connecting an AHB Lite master to a full AHB slave:

- Feed the slave's HREADY output signal back into its HREADY input to ensure proper timing and feedback.
- Also, connect the slave's HREADY output to the master.
- Tie the HSEL signal at the slave to logic high (1) to ensure the slave is always selected.

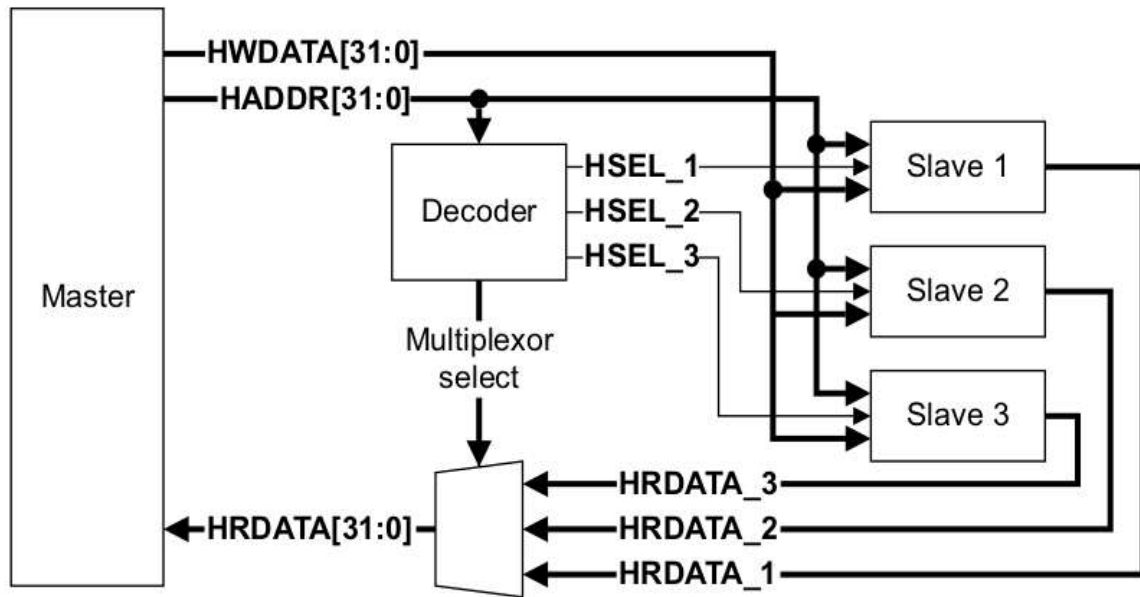


Fig.26: AHB-Lite block diagram

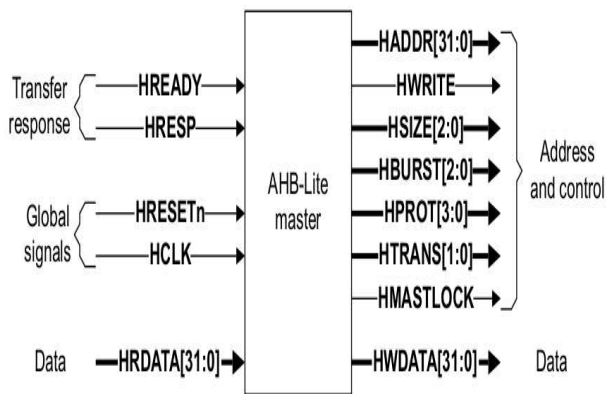


Fig.27 : Master interfaces

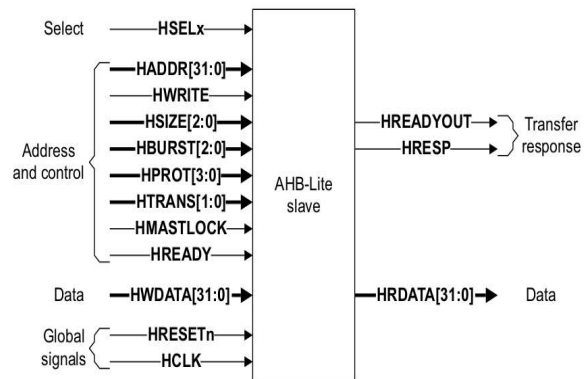


Fig.28 : Slave interfaces

Features of Protocol

1. Transfer Basics:

- A transfer starts when the master sets the address and control signals.
- Control signals include details like the address, transfer direction (read/write), data width, and burst type.
- Transfers can be:
 - Single: One transfer at a time.
 - Incrementing Bursts: Addresses increase without wrapping.
 - Wrapping Bursts: Addresses wrap around at predefined boundaries.

2. Data Buses:

- Write Data Bus: Sends data from the master to the slave.
- Read Data Bus: Sends data from the slave to the master.

3. Transfer Phases:

- **Address Phase:** The master sets the address and control signals (1 cycle only).
- **Data Phase:** Data is transferred and may take multiple cycles. Slaves cannot extend the address phase but can extend the data phase using the HREADY signal.

4. HREADY Signal:

- If a slave needs more processing time or response, HREADY is set to LOW.
- This adds wait states, giving the slave extra cycles to complete its operation.

5. HRESP Signal:

- Slaves use this signal to indicate the outcome of a transfer:
 - Successful transfer.
 - Error (if something went wrong).

Decoder and Multiplexor

1. Decoder:

- Identifies the address of each transfer.
- Activates the appropriate slave by generating a select signal.

2. Slave-to-Master Multiplexor:

- Combines responses and reads data from multiple slaves.
- Routes the correct response/data from the selected slave to the master.
- Controlled by the decoder to ensure proper data flow.

Advantages

1. Reduced Latency - reduces extra clock cycles by eliminating unnecessary signals like HBUSREQ and HGRANT.
2. Improved Performance - supports concurrent communication, making the system faster and more efficient.

Comparison

Feature	APB	ATB	AHB	AHB-Lite
Purpose	Low-bandwidth peripherals	Debugging and tracing	High-performance components	High-performance single-master systems
Performance	Low	N/A (debug focus)	High	High
Pipelining	N/A	N/A	Yes	Yes
Burst Transfers	N/A	N/A	Yes	Yes
Masters Supported	AHB-APB bridge	N/A	Multiple	Single
Complexity	Low	Medium	High	Medium
Use Case	Used for peripherals such as UARTs, GPIO	Debugging	Memory, DMA, processors	Single master system

Contribution:

1. AMBA - Ankit Kumar Pal and Nakul Garg
2. APB - Anshu Kumar
3. ATB - Nakul Garg
4. AHB - Shivam Shukla
5. AHB Lite - Ankit Kumar Pal
6. Comparison - Anshu Kumar and Dora Sai Kumar

References:

1. ARM, “AMBA 3 AHB-Lite Protocol® v1.0 Specification”, available at <http://www.arm.com>.
2. ARM, “AMBA APB Protocol® Version: 2.0 Specification”, available at <http://www.arm.com>.
3. ARM, “AMBA ATB Protocol Specification,” available at <http://www.arm.com>.
4. ARM, “AMBA Specification (Rev 2.0)”, available at <http://www.arm.com>.
5. Joseph Yiu “System-on-Chip Design with Arm Cortex-M Processors”, ARM Education Media, 2019 ISBN – 978-1-911531-18-0.